

PRESENTATION

PROJET DATA QUALITY

Yuxuan CHEN – Abdrahamane Mbourou CAMARA – Gide Azangue FOMAT – Hongxiang LIN – Liya XU

SOMMAIRE

01 Introduction & Présentation
des Jeux de Données

02 L'Approche Classique
(Pipeline Algorithmique)

03 L'Approche Agentique &
Choix du Workflow

04 Démonstration

05 Résultats Expérimentaux &
Comparatifs

06 Limites and conclusion

INTRO & PRÉSENTATION DES JEUX DE DONNÉES

- Les FDs classiques échouent sur les données réelles car les valeurs sont souvent bruitées ou uniques.
- Les PFDs approximatives travaillent au niveau des patterns
 - préfixes, suffixes, tokens, sous-chaînes
 - avec des seuils de support et de confiance pour tolérer les erreurs.

01

t1.csv

9 101 lignes

Données RH — noms d'employés, genres, départements.
Idéal pour les patterns tokens :
extraire le prénom pour prédire le genre.

t2.csv

3 502 lignes

Adresses d'entreprises
Parfait pour les patterns
sur les codes postaux.

t3.csv

1 077 lignes

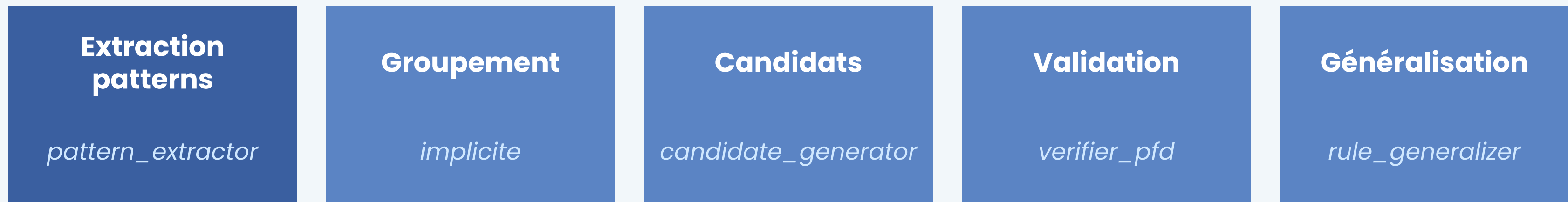
Licences de commerces au Maryland.
Même structure géographique que t2.
Permet de vérifier la généralisation de nos algorithmes.

On a écarté les fichiers ADN/protéines : trop spécialisés pour des LLMs généralistes.

APPROCHE CLASSIQUE

02

Pour découvrir les PFDs automatiquement, on a implémenté le pipeline classique du cours, piloté par la fonction centrale **discover** qui enchaîne 5 étapes.



→ **Sortie : liste des PFDs valides avec leur confiance**

*L'entrée c'est le dataset brut — on ne sait pas à l'avance quelles règles existent.
C'est l'algorithme qui les découvre en parcourant les 5 étapes.*

- *pdf_verifier* — Vérifie qu'une PFD candidate satisfait le seuil de confiance
- *candidate_generator* — Produit les paires $(X, \lambda_X) \rightarrow (Y, \text{top}_Y)$ à tester
- *pattern_extractor* — Extrait les motifs candidats de chaque colonne
- *rule_generalizer* — Fusionne les règles spécifiques et élague les dominées

ÉTAPES 1 & 2

02

Étape 1 —extraction des patterns : *pattern_extractor*

Ce module parcourt chaque colonne et extrait les patterns. Dans notre implémentation on a retenu quatre types :

Préfixes

Pour une valeur '60601' → génère '6', '60', '606', '6060'...
→ Utiles pour les codes postaux, identifiants.

Tokens

Pour 'John Smith' → extrait 'John' (premier mot).
→ Utiles pour les noms, titres de postes.

Suffixes

Pour 'user@example.com' → extrait '@example.com'.
→ Utiles pour les emails, extensions.

Sous-chaînes

Recherche de mots-clés métier.
→ Ex : 'Police' dans un titre de poste.

*Chaque pattern est stocké avec les indices des lignes qui lui correspondent.
Cela évite de re-scanner la colonne à l'étape suivante*

Étape 2 — Groupement

Le groupement est implicite dans notre implémentation.

✓ Chaque pattern garde directement les indices des lignes qui lui correspondent
le groupe est déjà formé.

Pas d'étape séparée nécessaire.

Exemple

"606" → [ligne 1, ligne 2, ligne 5, ligne 8...]
"60" → [ligne 1, ligne 2, ligne 5, ligne 12...]
"6" → [toutes les lignes ZIP en 6...]

ÉTAPES 3 & 4

02

Étape 3 — candidate_generator

Pour chaque pattern sur une colonne X et chaque colonne cible Y, le module prend le groupe de lignes correspondant et cherche la valeur la plus fréquente de Y dans ce groupe.

*Exemple : dans le groupe où ZIP commence par "606", la valeur la plus fréquente de CITY = "Chicago"
→ Candidat créé : ZIP startswith "606" → CITY = "Chicago"*

on génère un candidat pour chaque combinaison possible de pattern et de colonne cible. Cela peut représenter plusieurs milliers de candidats sur un dataset réel.

⚠ C'est la principale limite : exhaustif et sans sémantique.

Des milliers de candidats dont beaucoup sont trivaux.

Étape 4 — verifier_pfd

C'est la brique centrale du pipeline. Pour chaque candidat, elle calcule deux métriques :

Support

Nombre de lignes où X matche le pattern.

Confiance

Parmi ces lignes, quelle proportion vérifie aussi Y ?

Si confiance $\geq 1 - \epsilon$ → la règle est valide

Exemple concret :

*ZIP startswith "606" → CITY = "Chicago"
3 000 lignes matchent ZIP. 2 940 ont CITY = "Chicago".
Confiance = 98% → avec $\epsilon = 0.1$, la règle est valide ✓*

Étape 5 — rule_generalizer

Sans cette étape, le pipeline produit beaucoup de règles redondantes.

Par exemple on peut trouver simultanément :

- *ZIP startswith "606" → Chicago*
- *ZIP startswith "6060" → Chicago*
- *ZIP startswith "60601" → Chicago*

Le module procède en deux temps :

Fusion

Regroupe les règles qui partagent la même cible et calcule leur préfixe commun → "606". Vérifie que la règle fusionnée est toujours valide.

Élagage

Si "606" → Chicago existe déjà, les règles plus spécifiques comme "6060" sont supprimées car dominées.

Limite de l'approche

Cette approche est exhaustive et purement syntaxique.

Elle explore mécaniquement des milliers de candidats sans comprendre la sémantique des données.

Cela génère beaucoup de règles triviales ou peu intéressantes.

→ Ce qui motive l'approche agentique

Au lieu d'explorer tout mécaniquement, on laisse le LLM guider la recherche vers les candidats sémantiquement pertinents.



03

APPROCHE AGENTIQUE – POURQUOI LE WORKFLOW 2 ?

NOTRE CHOIX

WORKFLOW 1

Feature-Enriched Discovery

L'agent suggère des transformations, mais l'algorithme explore ensuite tous les candidats possibles sans filtre sémantique.

~15 000 candidats testés malgré tout

Pas assez efficace

WORKFLOW 2

Guided Search

L'agent guide les **transformations** et priorise les **candidats $X \rightarrow Y$** . L'algorithme valide uniquement les candidats sélectionnés.

~15 candidats au lieu de 15 000 (×1000)

Meilleur compromis qualité / efficacité

WORKFLOW 3

Agent-in-the-Loop Discovery

L'agent propose des règles, analyse les violations et **raffine** ses hypothèses à chaque cycle – boucle itérative avec l'algorithme

Plus puissant – complexité d'implémentation ×3

LLM stable requis

03

APPROCHE AGENTIQUE – SÉPARATION DES RÔLES

Agent LLM

**Guidance sémantique –
suggère les transformations
et priorise les paires $X \rightarrow Y$
prometteuses**

ZIP, CITY => liés géographiquement

**Department, Division
=> logique organisationnelle**

Algorithme

**Validation rigoureuse –
calcule support et confiance
sur les données réelles**

support

confiance

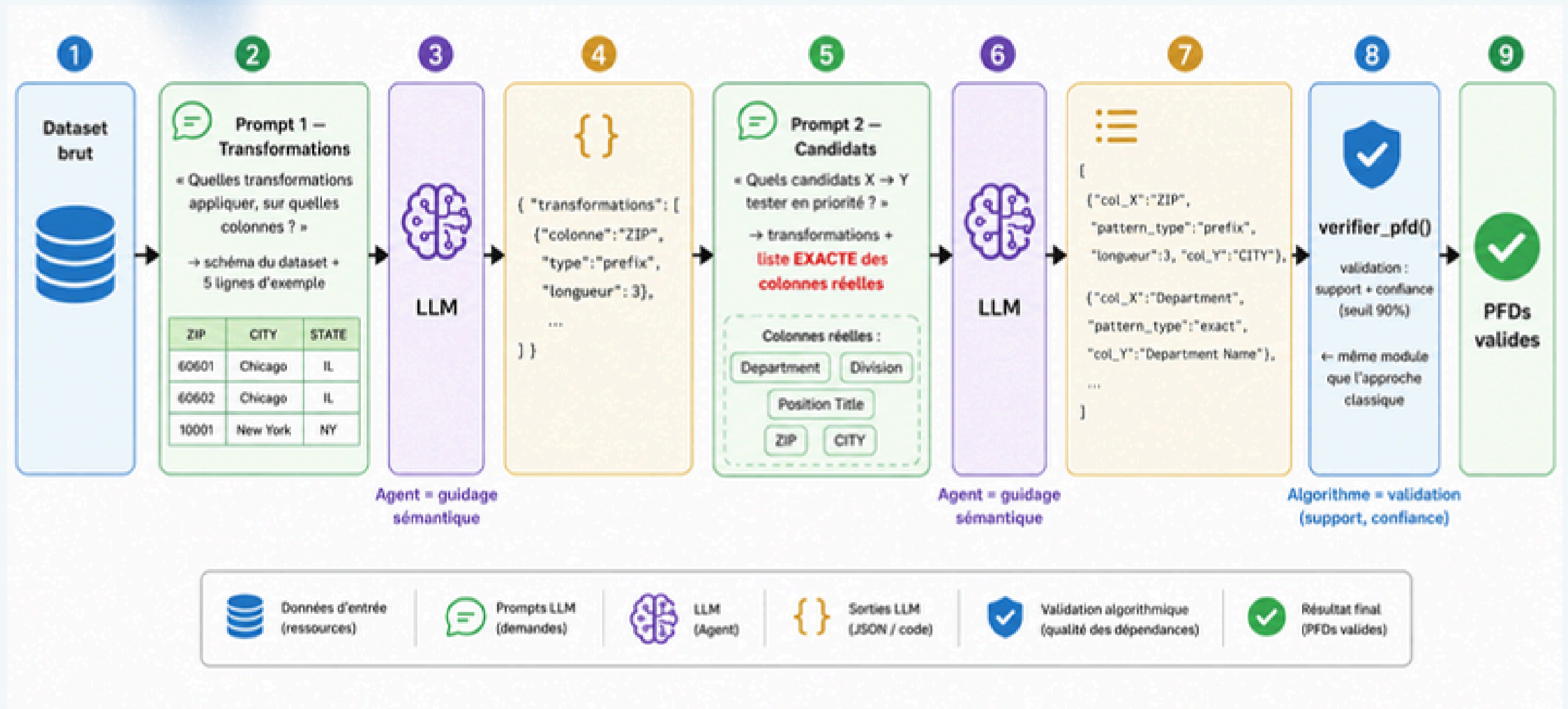
seuil

Aucun des deux ne peut faire le travail de l'autre.

03

APPROCHE AGENTIQUE – RECHERCHE GUIDÉE

Pipeline Workflow 2



04

DÉMONSTRATION

05

- **Exhaustive** : couverture large mais seulement beaucoup sont triviales, et peu pertinentes
- **LLMs** : peu de règles mais très ciblées (80-100 % pertinentes)

→ **Arbitrage couverture vs qualité**

L'exhaustive sert de filet large, les LLMs de filet ciblé

- Groq rapide sur t1/t2, **échec sur t3**
- DeepSeek 8B meilleur sur t3, mais très lent
- **Hallucination - Prompt**

COMPARAISON GLOBALE

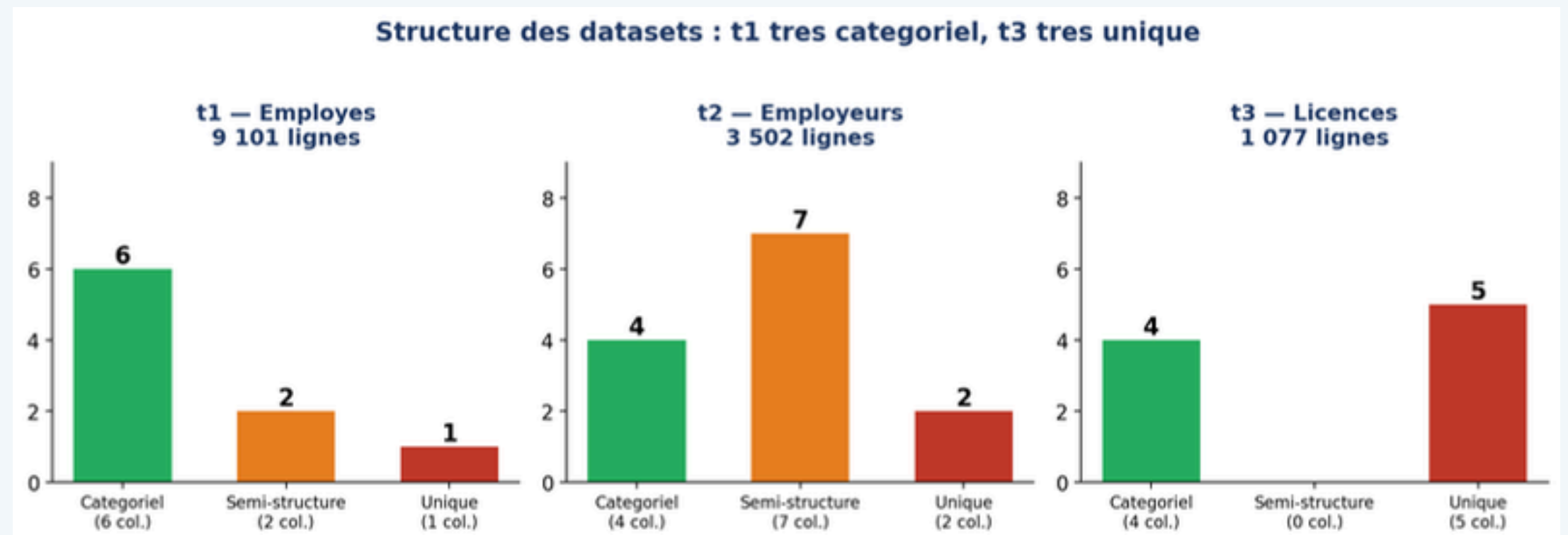
Dataset	Méthode	PFDs	Candidats testés	Temps
t1	Classique	870	~15 000	33 s
t1	Guided + Groq	107	12	2,2 s
t1	Guided + DeepSeek 1.5b	0	3	38 s
t2	Classique	2 608	~23 000	56 s
t2	Guided + Groq	128	15	3,8 s
t2	Guided + DeepSeek 1.5b	213	16	2,5 s
t2	Guided + DeepSeek 8b	227	15	479 s
t3	Classique	271	~5 400	11 s
t3	Guided + Groq	0	2	49 s
t3	Guided + DeepSeek 1.5b	0	3	58 s
t3	Guided + DeepSeek 8b	108	15	495 s

05

STRUCTURE DES DATASETS

- **t1** : 88 % pertinentes – les colonnes **catégorielles** produisent des PFDs de qualité
- **t2** : 31 % – **COUNTRY**="United States" (99,7 %) et **ACTIVE**="Y" (97 %) polluent
- **t3** : 20 % – **State**="MD" (1 seule valeur dans toute la colonne)

→ Plus le dataset est catégoriel, plus la pertinence est haute



05

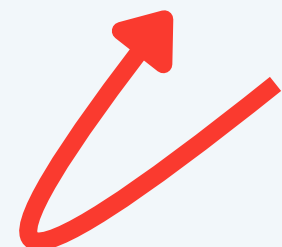
ANALYSE DES PFD

Exemples de règles découvertes par le guided search sur t1

```
Department['POL'] → Department Name='Department of Police'    (conf 100%,  
sup 1835)  
Department['HHS'] → Department Name='Department of Health and Human  
Services'    (conf 100%, sup 1512)  
Division['MSB'] → Assignment Category='Fulltime-Regular'    (conf 100%, sup  
287)
```

Exemples de règles trouvées par la méthode classique sur t2 :

```
FAX → CITY  
PHONE → EMPLOYER_ID  
ADDRESS_2 → CREATED_DATE
```



Statistiquement vraies ≠ utiles métier

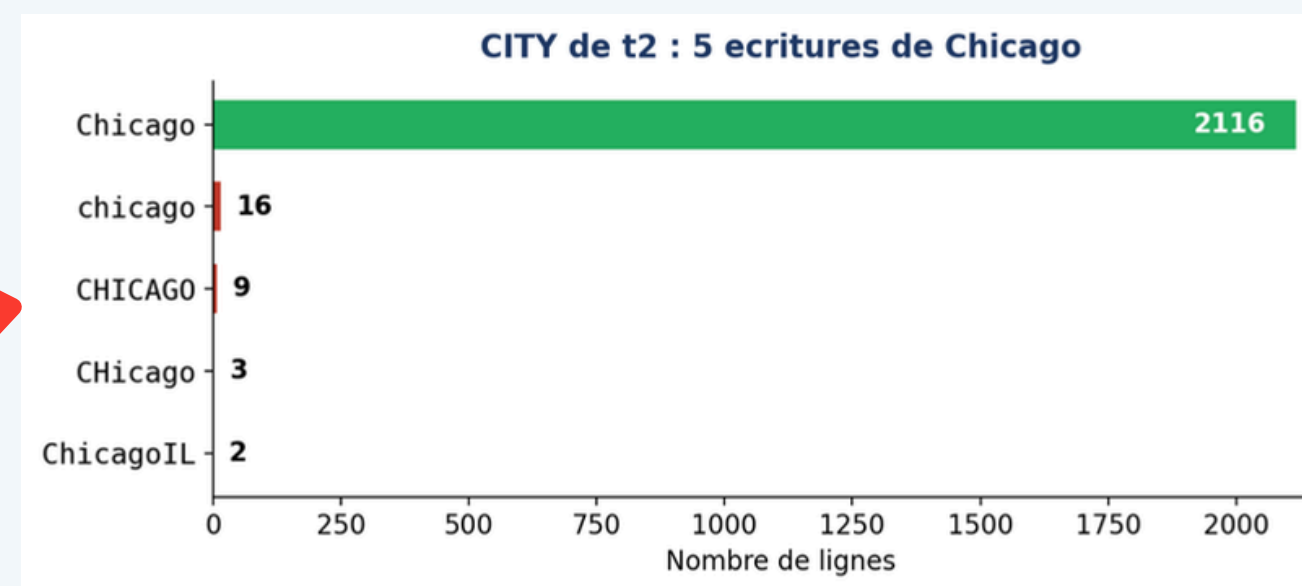
05

PFDS COMME OUTIL DE DIAGNOSTIC DE QUALITÉ

DATASET T2

col_X	pattern_X	match_type	col_Y	pattern_Y	confidence	support	violations	trivial	raison_trivial
NAME	LLC	endswith	COUNTRY	United States	1.0	607	0	True	valeur_dominante
NAME	LLP	endswith	COUNTRY	United States	1.0	337	0	True	valeur_dominante
CITY	New York	exact	STATE	NY	1.0	267	0	False	
ADDRESS_2	Ste.	startswith	COUNTRY	United States	1.0	141	0	True	valeur_dominante
NAME	, LLP	endswith	COUNTRY	United States	1.0	83	0	True	valeur_dominante
STATE	Il	exact	COUNTRY	United States	1.0	54	0	True	valeur_dominante
ADDRESS_1	Clark St.	contains	CITY	Chicago	1.0	43	0	False	
STATE	IL	exact	COUNTRY	United States	0.998771	2441	3	True	valeur_dominante
ADDRESS_2	Suite	startswith	COUNTRY	United States	0.996364	275	1	True	valeur_dominante
ADDRESS_1	LaSalle	contains	CITY	Chicago	0.994286	350	2	False	
ZIP	60601	exact	CITY	Chicago	0.99022	409	4	False	
ZIP	606	startswith	CITY	Chicago	0.982637	2131	37	False	
ADDRESS_1	Dearborn	contains	CITY	Chicago	0.973684	11	3	False	
FAX	-312	startswith	CITY	Chicago	0.970007	1367	11	False	
ADDRESS_2	Ste	startswith	COUNTRY	United States	0.969231	65	2	True	valeur_dominante
ZIP	60610	exact	CITY	Chicago	0.959459	74	3	False	
PHONE	-312	startswith	CITY	Chicago	0.955493	1775	79	False	
FAX	-630	startswith	STATE	IL	0.949367	79	4	False	
PHONE	-773	startswith	CITY	Chicago	0.92973	185	13	False	

- La PFD ZIP startswith "606" → CITY = "Chicago" a des violations
 - En investiguant : 30 ne sont pas de vraies violations, juste de la casse
 - chicago (16×), CHICAGO (9×), CHicago (3×)
 - Même problème sur STATE : IL / Il / IL et sur PHONE et FAX
- => Les **PFDS signalent les problèmes** de qualité sans qu'on ait à les chercher



NOS 3 FILTRES ANTI-HALLUCINATION

1

MOTIF TROP COURT

Un pattern d'un seul caractère est trop générique pour être discriminant — il couvre trop de valeurs sans révéler de structure réelle.

- *startswith("A")* → rejeté
- *startswith("Maryland")* → ✓ conservé

→ Élimine les suggestions superficielles des LLMs

2

VALEUR DOMINANTE > 95%

Si une colonne est quasi-uniforme, toute PFD vers elle est vraie par défaut — non pas parce qu'il y a une règle, mais parce que le dataset est homogène.

- Colonne "Pays" : 99 % "USA" →
PFD X → Pays : triviale

→ Évite les fausses dépendances liées aux données

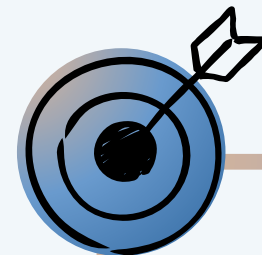
3

COLONNE MONOVALENTE

Une colonne avec une seule valeur distincte ne peut pas être la cible d'une dépendance utile — la règle est vraie mais ne dit rien.

- Colonne "Version" = "v1" partout
→ *Toute PFD vers elle : rejetée*

→ Filtre les tautologies sans valeur sémantique



IMPACT CONCRET

t2 : 1 905 règles triviales éliminées (69%).
t3 : 506 règles triviales éliminées (80%).
Sans ces filtres, les résultats LLM sont inexploitable.

PFDS + LLMS : UN DUO PROMETTEUR, MAIS ENCADRÉ

Ce qu'on a prouvé

Les LLMs accélèrent la découverte de PFDs et ciblent les règles sémantiquement riches — là où l'exhaustif noie dans le bruit.

Apport de l'approche hybride

Guided search pour la pertinence, énumération classique pour la couverture — les deux sont complémentaires, pas concurrents.

Limites confirmées

La taille du modèle prime sur la vitesse. Sans filtres anti-hallucination, les petits LLMs inventent colonnes et patterns.

Perspectives

Fine-tuning sur des schémas réels, vote multi-LLM, extension aux patterns sémantiques au-delà des préfixes.

Ni l'exhaustif seul, ni le LLM seul ne suffisent — c'est leur combinaison qui maximise qualité et couverture.



THANK YOU